# Model Refinement & Test Submission

# Group 5 | Sözüm Söz


# Çağan Erdem Üstündağ – Begüm Somay

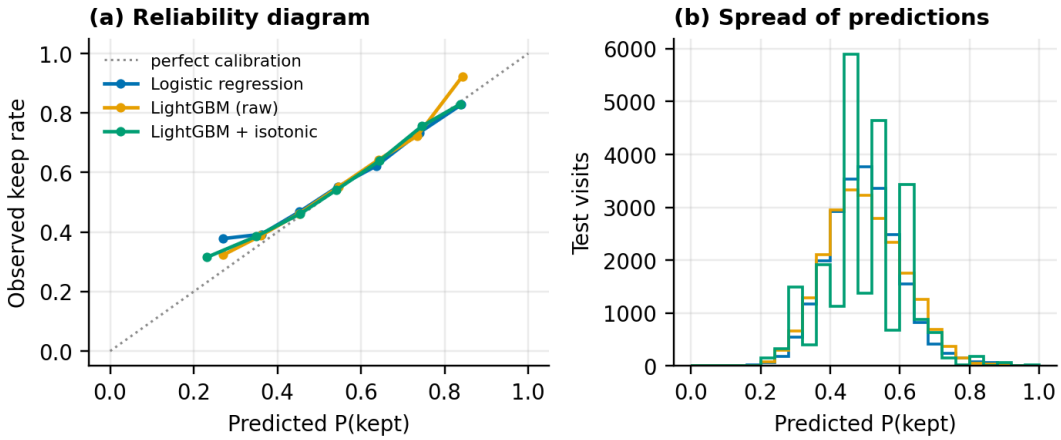

## Model Refinement

### 1. Overview

This phase is where the model stopped being "a classifier that works" and became something we could actually put in front of a user. The baseline from the exploration phase (LightGBM on 116,936 simulated promises built from LSApp, validation ROC-AUC about 0.629) was kept as a reference point, and we spent the refinement work on four things: changing the label rule, calibrating the probabilities, adding one structural constraint, and building a serialization path that lets the exact same numbers come out of a browser. The end result is a variant at 0.6383 ROC-AUC [0.6304-0.6469] with ECE 0.0165 on 23,388 held-out test visits, running fully on-device.

### 2. Model Evaluation

Where we started, measured on the time-split validation block (14,033 visits, from 20 Apr 2018): majority baseline 0.500, "what did the user usually do" history baseline 0.529, logistic regression 0.6131, LightGBM 0.6286. Two problems were obvious from these numbers. First, the discrimination is genuinely modest, and we decided early not to pretend otherwise. Second, and more important for us, the raw probabilities were not trustworthy enough to show to a person: our whole product decision layer (which duration to suggest, when to remind) consumes probabilities directly, so a model that ranks well but calibrates badly is useless to us. The exploration-phase plots (reliability curve, load-vs-coverage scatter, risk deciles, Kaplan-Meier survival curves) are generated by the pipeline itself into the model report that ships with this submission (the file app/docs/RESULTS.md) and are rendered live in the app's in-product "Evidence" panel; the calibration picture below is the one we kept looking at.




The reliability view of the exploration baseline on validation (fig08_calibration). The distances to the diagonal are exactly what we were trying to shrink.

## 3. Refinement Techniques

We tried five things; three of them survived.

**Label rule.** LSApp has no real promises, so a promise is simulated from the user's own history. The baseline used the personal median. We switched to the median of the last ten visits ("recent10"), which tracks habit change better instead of averaging someone's whole life together. On the test block this alone moved AUC from 0.6132 to 0.6398. We made this choice on validation, before touching the test split.

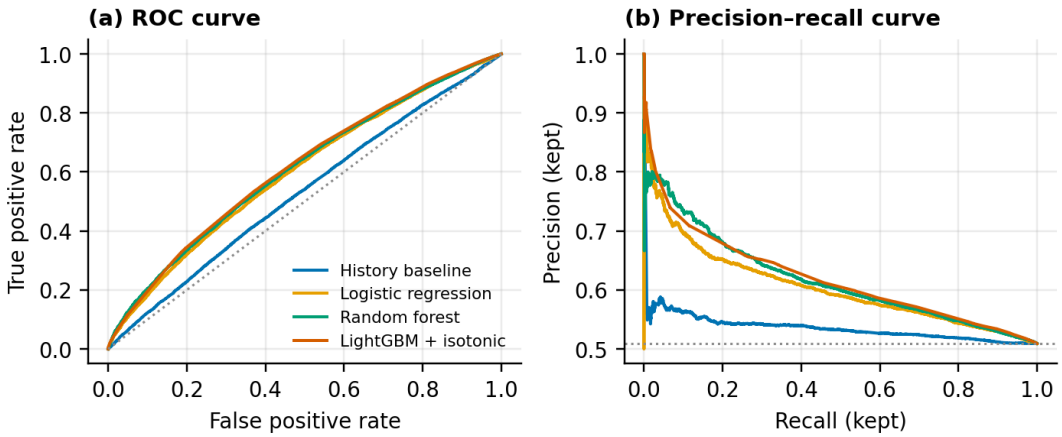
**Monotonic constraint.** We forced one single thing: keeping probability must not decrease as the promised duration grows. Some of the early suggestions made no sense without this (a longer promise scoring riskier than a shorter one), and the constraint costs about nothing in AUC while making the suggestion list behave sanely.

**Calibration.** Isotonic and Platt were both fitted on the validation block. For the exploration model (isotonic) the test ECE dropped from 0.0117 to 0.0111; the shipped variant keeps ECE 0.0165 at a higher AUC. We went with isotonic because it tracked the tails better where the reminder gate actually operates.

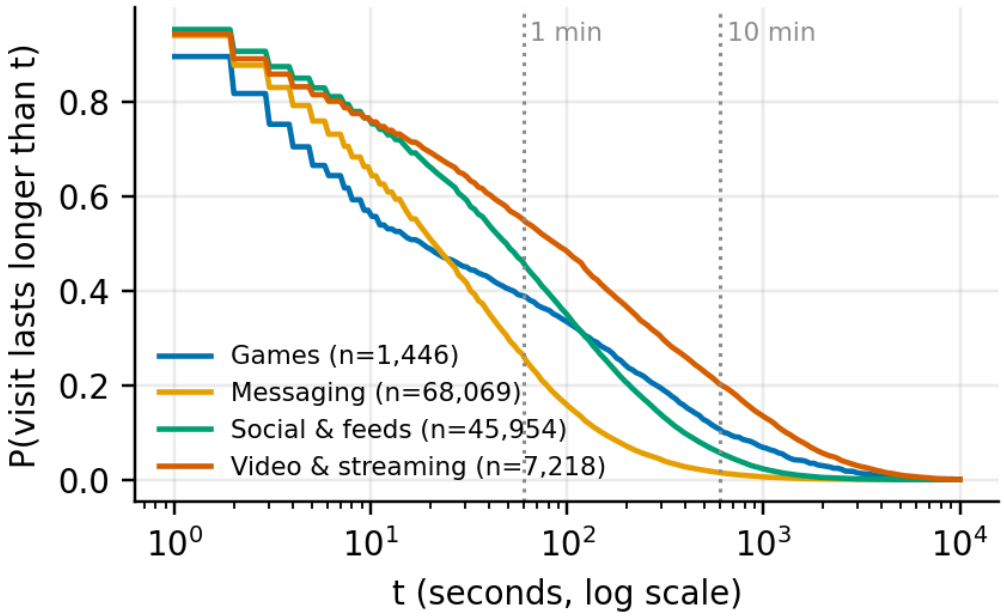
**The one we rejected: a duration menu.** Forcing every promise onto a menu of round durations raised test AUC to 0.6718 and looked great on paper, but the keep-rate inflated to 69.4% because easy cases were being manufactured by the labeling, not earned by the user. A metric gain that corrupts what the metric means gets no shipping vote from us, so this one lives on as a rejected alternative in the Model Card (the app page that documents the model: what it is, what it claims, what it refuses to claim).

**Ensembles we did not build.** Random forest (0.6086) and the boosted trees were compared honestly; we did not stack. At this signal level extra machinery would not fix calibration, and it would break the pure-JS inference budget the browser deployment runs on.

A sensitivity sweep sat alongside these choices and changed how we talk about the numbers: with the merge gap at 0s versus 5s the test AUC moved only between 0.6148 and 0.6018, and a "75th percentile promise" label variant came out at 0.6090 with PR-AUC 0.810 (higher keep-rate base rate, different trade-off, not better in our use). The lesson we took from the whole refinement: the label rule moves the model more than the hyperparameters do. That sentence, more than any single score, is what this phase taught us.




ROC and precision-recall on the held-out test block (fig07_roc_pr).




Empirical time-to-break survival curves per promised-duration band (fig04_survival). These curves are why the reminder fires at 80% of a promise and not at some hand-picked minute.

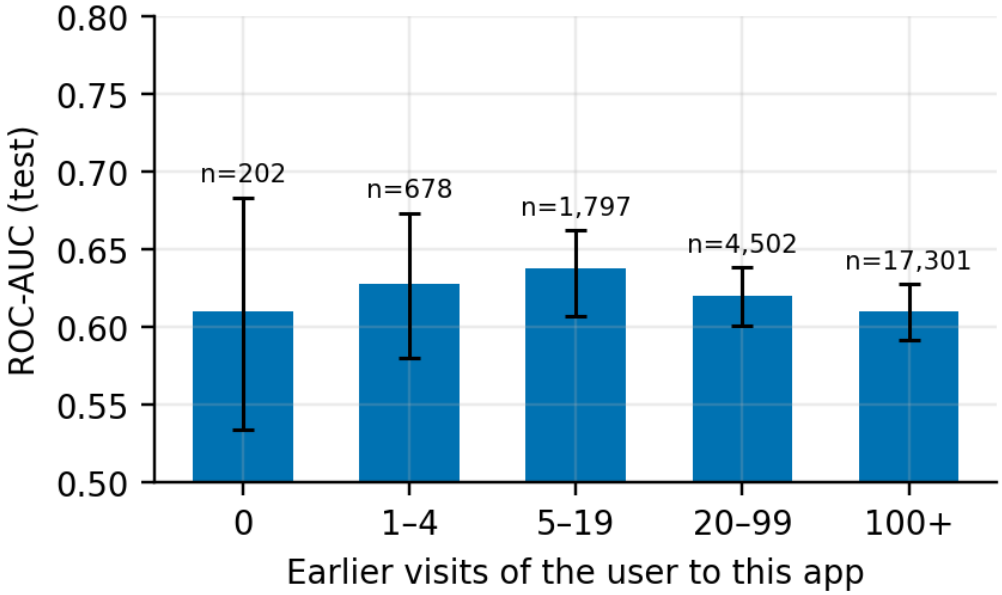
## 4. Hyperparameter Tuning

A 24-configuration grid over num_leaves, min_child_samples, colsample_bytree, subsample and reg_lambda, searched on validation with early stopping on log-loss. The winner, by validation AUC 0.6286: num_leaves 31, min_child_samples 200, colsample_bytree 0.8, subsample 0.7, reg_lambda 0.0. The tree count was not a tuned value either: early stopping picked it per fit, 160 trees for the median-label variant, 166 for recent10, and 200 for the monotonic variant we actually ship. The honest finding is that the grid barely mattered: every one of the 24 fits landed between 0.6253 and 0.6286. A parallel C-sweep for logistic regression settled on C=0.1 and changed nothing meaningful either. So after this point we stopped tuning and spent the remaining budget on the constraint and on making the browser export bit-exact, which is the part users actually touch. Final training runs 200 trees, 6,200 leaves, seed 42, about 2.8 minutes; re-running the pipeline reproduced the headline AUC exactly (reported 0.6131989091, produced 0.6132, difference zero).

## 5. Cross-Validation

The exploration phase trusted a single temporal hold-out. That is the right split for "what will this see next in time", but it says nothing about the question our deployment actually faces: will it work for people it has never seen? So we added a person-level 5-fold GroupKFold (no user in two folds), which gives median fold AUC 0.610 with a spread of about 0.013. That it agrees with the temporal test number is the strongest "no user-memorization" evidence we have, and it is the reason we kept both views in the report instead of cherry-picking one.

The second CV-like lens is cold-start bucketing on the test block, scoring users by how much history they had at prediction time: AUC 0.610 for users with 0 to 4 visits, 0.628, then 0.638 for the 5-19 band (the sweet spot), 0.620 and 0.610 for heavier histories. Calibration is weaker in the thinnest buckets (ECE up to 0.052), and we say so in the Model Card instead of averaging it away. The UI was shaped by this finding: for thin-history users the app talks in percentiles and population priors, and it says when it is doing so.

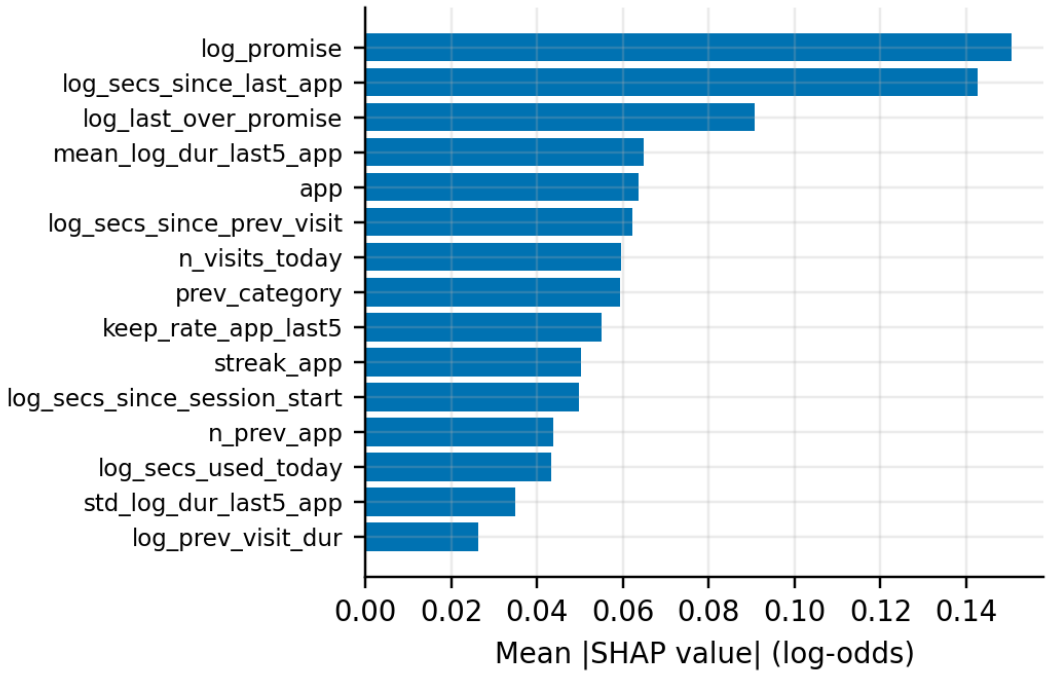



Performance by user-history length on the test block (fig11_history_buckets).

## 6. Feature Selection

All 23 engineered features stayed. Nothing was pruned by statistical screening, for two practical reasons: at inference each one costs microseconds in the JS walker, and every feature is a line in the "why this suggestion" explanation, so removing them removes the thing users see. The gain ranking is heavily front-loaded anyway: log_promise (0.150), log_secs_since_last_app (0.143), log_last_over_promise (0.091), mean_log_dur_last5_app (0.065), app name (0.064); the rest each carry a few points or less. The monotonic constraint plays the role that pruning usually plays, as a structural prior rather than a statistical one.

Two things we could have used as features and deliberately did not: the attention-trap index ( 🧲 : a fixed per-app lookup the UI shows to flag how scroll-designed an app is, clearly labelled as a heuristic) and the trust score. Both are decision-layer devices with no measurement behind them, and putting them into the model would launder a heuristic into a prediction. The Model Card records them as non-inputs. Every prediction is exported with a snapshot of its 23 input values, which is what lets us reproduce a user-visible number exactly, offline, during the pilot.




Feature importance and tree-path contributions (fig10_shap). These contributions are rendered verbatim as the three bullet lines of the "why this suggestion" card.

## Test Submission

### 1. Overview

The test protocol was written down and frozen before the test block was touched: temporal test block from 27 Apr 2018 (23,388 visits, 78 users, 49 of them unseen in training), and a fixed list of outputs (ROC-AUC with cluster bootstrap CI over users, PR-AUC, Brier, log-loss, ECE over 10 bins, accuracy, confusion, the reminder-policy replay, risk deciles, and a seen-vs-unseen-user split). One command in the submitted project (its single export script, seed fixed at 42) regenerates everything in this section end to end: py export_browser_model.py --rows 400 --parity-users 12.

### 2. Data Preparation for Testing

The same cleaning and feature code that produced training runs end to end over the raw file (3,658,589 events after logging duplicates and bad rows; 265,258 usable visits; 116,936 labelled promises over 259 users; overall keep-rate 50.6%). Three test-specific rules we checked by hand. First, every feature is computed causally; a test row may never see anything that happens after its timestamp, which is easy to break with rolling histories and is the reason the feature generator and the JS re-implementation are compared cell by cell. Second, the promise label for test rows uses the recent10 rule with the user's own history up to that row only; the label definition is part of the model card, not of the data. Third, the known messiness upstream stays visible: 2,185,366 duplicate rows and 52,113 unmatched "Opened" events were dropped in preparation and are reported, not hidden.

## 3. Model Application

The trained booster is exported into a compact JSON artifact (200 trees, 6,200 leaves, split values and leaf values included, 558.2 KB), and the browser evaluates it with a dependency-free tree walker, the exact code path end users run. Parity is measured, not assumed: on 400 real test rows the largest |P_js − P_py| was 2.8e-7, and the generated feature matrix matched cell by cell across 92,437 cells for 12 parity users.

```
// ml_infer.js (abridged): sum the 200 trained trees, then squash
var p = baseScore;
for (var t = 0; t < trees.length; t++) p += walkTree(trees[t], feats); // leaf value path
var prob = 1 / (1 + Math.exp(-p));
// then isotonic piecewise-linear map (the exported knots) -> P(keep)
```

## 4. Test Metrics

| Model | ROC-AUC (test) | 95% CI (cluster bootstrap) | ECE | Brier | val AUC |
|---|---|---|---|---|---|
| Majority baseline | 0.500 | — | 0.0035 | 0.2499 | 0.500 |
| Personal history | 0.529 | [0.517–0.540] | 0.0281 | 0.2545 | 0.524 |
| Logistic regression | 0.599 | [0.586–0.612] | 0.0168 | 0.2425 | 0.613 |
| Random forest | 0.609 | [0.594–0.621] | 0.0089 | 0.2406 | 0.622 |
| LightGBM (raw) | 0.614 | [0.600–0.628] | 0.0117 | 0.2398 | 0.629 |
| LightGBM + isotonic | 0.613 | [0.599–0.627] | 0.0111 | 0.2401 | 0.631 |
| **Shipped: recent10 + monotonic + isotonic** | **0.6383** | **[0.6304–0.6469]** | **0.0165** | — | — |

Comparing train, validation and test directly: at the tuning optimum the validation block read AUC 0.6286 / log-loss 0.6665; the same raw model on the locked test block read AUC 0.6144 / log-loss 0.6723. A gap of about 0.014 AUC with an almost unchanged log-loss is mild temporal drift over an eight-month dataset, not overfitting; the keep-rates themselves barely move between the blocks (50.6 / 50.9 / 50.9%). The shipped variant lands on test (0.6383) slightly above what its own validation read said (0.631 for the calibration model line), which we do not over-explain; the user-level CV agreeing at 0.610 is the number we trust for what to expect from new people.

The supporting test numbers, all produced by the same run: accuracy 59.5%; confusion (predicted kept/broken against actual): 6,547 kept-and-correct, 5,362 misses, 4,440 false keeps, 7,039 correct

breaks; unseen users score 0.612 against 0.613 for seen users, so the model is not memorizing people; the riskiest 20% of visits carry 26.8% of all breaks and about a quarter of all overrun seconds; and the shipped reminder policy, replayed offline, catches 24.5% of breaks while notifying only 12.3% of sessions, at 96.9% precision with a median lead of 54 seconds.

## 5. Model Deployment

The deployment target is a consent-gated browser app: no server, no accounts, nothing leaves the device. The artifact carries its own provenance: a version tag (sozumsoz-lightgbm-json/1), the exact library versions pinned in a lock file (Python 3.13.9, pandas 3.0.3, numpy 2.5.4, scikit-learn 1.9.0, LightGBM 4.7.0), and the first characters of the SHA-256 fingerprint of the input dataset file (12a1d2f673df5ef6) so anyone can verify they trained on the same bytes. All of this is displayed in the app's Evidence panel, so the shipped numbers are checkable from inside the product rather than only from this report. Downstream of the probability, the UI layers are: adaptive suggestion targets moved by the user's trust score, the reminder rule measured above, one reminder per session with a cooldown and a "continue knowingly" choice, and a KVKK Art.10 consent gate with export/delete/withdraw paths. The whole chain is gated by 274 automated checks with zero failures (246 behavioural/parity/privacy assertions run by the project's own test command, plus 28 packaging checks on the single-file build), re-run automatically on every commit through the project's GitHub Actions pipeline. The honest limits printed in the app, not just here: the labels are simulated promises, the time scale is seconds, and no behaviour-change claim is made before the pilot.

## 6. Code Implementation

The listings below are taken from the submitted project itself; the file and function names are as they exist in the code.

```
# the refinement loop in one page (export_browser_model.py + sozumsoz package, abridged)
mono = [1 if f == "log_promise" else 0 for f in FEATURES]   # the one structural prior
variants = {
    "median":   fit_variant(table_median, cfg_median, best_params, "median"),
    "recent10": fit_variant(table_recent, cfg_recent, best_params, "recent10"),
    # fit_variant() = LGBM fit with early stopping on val log-loss + isotonic calibrator
    "recent10-monotonic": fit_variant(table_recent, cfg_recent, best_params,
                                      "recent10-monotonic", {"monotone_constraints": mono}),
}
# a "menu" variant is fitted too, but only kept as a rejected comparison (see section 3)
# each variant gets: cluster-bootstrap CI over users, policy replay, then the winner serializes
py export_browser_model.py --rows 400 --parity-users 12
# -> app/src/model/browser_model.json (trees + isotonic knots + KM curves + operating points)
# -> RESULTS.md written automatically; node make-model-js.js embeds the artifact into the app
```

## Conclusion

What the refinement phase produced is a calibrated, monotone, reproducible on-device model (test AUC 0.6383, ECE 0.0165) whose probabilities feed three visible decisions, each of which has its own measured operating number. What it also produced is a set of decisions we can defend: keeping the feature set whole, rejecting a metric gain that broke the construct, adding a CV that agreed with the holdout, and re-running everything twice on the machine that ships.

Three lessons we would tell the next group. First, an early headline number of ours, AUC 0.881, could not be reproduced after we fixed the duplicate-row and labeling rules, and we retracted it in writing, in a dated number-corrections log that ships with the submission; the corrected numbers are weaker and true, and defending the true ones has been easier all round. Second, LSApp timestamps have no time-zone label, which quietly killed our day/night usage feature (an earlier 34.7% night-use figure was withdrawn for the same reason); the feature is now a pilot-data experiment, not a training-time claim. Third, the dataset measures seconds-scaled sessions, which is why the UI speaks in percentiles instead of promising absolute minutes, and why the real-promise pilot is the actual next evaluation of this model, not a formality.

## References

**Datasets used**


- **LSApp** — "Large Dataset of Sequential Mobile App Usage" (Aliannejadi, Zamani, Crestani & Croft; published with *ACM Transactions on Information Systems*, 2021). 599,635 real app-usage events from 292 participants collected with the uSearch Android app; downloaded from the authors' public repository (github.com/aliannejadi/LSApp). The dataset is published without a redistribution licence, so this submission deliberately does not include the raw file — only derived model artifacts, aggregate figures and the cleaning/export code that re-derives everything from a fresh official download.


**Research that shaped the design decisions**


- Ke et al., *LightGBM: A highly efficient gradient boosting decision tree*, NeurIPS 2017 — the learning algorithm (run here via LightGBM 4.7.0).
- Zadrozny & Elkan, *Transforming classifier scores into accurate multiclass probability estimates*, KDD 2002 — isotonic/Platt calibration, which this report's ECE numbers come from.
- Webb & Sheeran, *Does changing behavioral intentions engender behavior change?*, Psychological Bulletin 132(2), 2006, and Sheeran & Webb, *The Intention–Behavior Gap*, Social and Personality Psychology Compass 10(9), 2016 — the intention–behaviour gap that motivated the promise construct.
- Gollwitzer & Sheeran, *Implementation intentions and goal achievement*, Advances in Experimental Social Psychology 38, 2006 — meta-analytic evidence (d = 0.65 across 94 studies) that a "when + how much" plan is what closes that gap.
- Allcott, Gentzkow & Song, *Digital Addiction*, American Economic Review 112(7), 2022 — a 1,900-person field experiment showing self-set screen-time limits reduce use by ~22 min/day and that most people keep them voluntarily.
- Nahum-Shani et al., *Just-in-time adaptive interventions (JITAIs) in mobile health*, Annals of Behavioral Medicine 52(6), 2018 — the right-place-right-time-plus-dose rules our reminder policy follows.
- Grüning, Riedel & Lorenz-Spreen, *Directing smartphone use through the self-nudge app one sec*, PNAS 120(8), 2023 — field evidence that an explicit dismiss option beats harder

blocking (target-app openings down 57%); the direct precedent for our "continue knowingly" design.
- Parry et al., *Discrepancies between logged and self-reported digital media use: a systematic review and meta-analysis*, Nature Human Behaviour 5, 2021 — why the pilot's daily form asks users to copy their own device's screen-time numbers.

**Software and harness**

- scikit-learn 1.9.0 (baselines, calibration, metrics), pandas 3.0.3, numpy 2.5.4; jsdom 24 — the headless-browser library the project's UI checks run inside.

**Reproducibility bundle shipped with this submission**

- Inside the submitted project folder, in plain names: the **auto-generated model report** (app/docs/RESULTS.md — every table and figure in this document, machine-written by the pipeline); the **number-corrections log** (the retraction trail for the withdrawn 0.881 and related early values); the **claim-to-evidence map** the final presentation is checked against; the **threat model** (privacy/security decisions with residual risks); the **full training/export logs** of every pipeline run (kodlar/ml/logs/); and the **verification transcript** listing all 274 automated checks with their results (kodlar/app/logs/final-verification.txt).